

PRESENTATION BRIEFING

Prolog Reasoning in AstroLogic

A complete walkthrough of the symbolic reasoning engine that powers zodiac, tarot, compatibility, and horoscope logic — how it's built, how it's called, and how it stays explainable.

AstroLogic · ~4,600 lines of Prolog across 12 modules · Prepared for presentation

1. Where Prolog Sits in the Architecture

The single idea to open with: **the LLM never reasons — it only phrases.** Every factual decision (which traits apply, which tarot cards are eligible, compatibility scores, which spread to recommend) is decided by Prolog. This is stated directly in the project's own docs:

FROM DOCS/AI-INTEGRATION.MD

"AstroLogic uses OpenRouter as the LLM provider for natural language generation. The AI is never responsible for core reasoning — it receives structured facts from Prolog and generates human-readable interpretations."

User Question



Python NLP (question classification)



Prolog (symbolic reasoning)



Structured Facts (zodiac, element, spread, cards, themes)



OpenRouter (LLM natural-language interpretation)



Final Reading (displayed in frontend)

If the LLM is unavailable, Prolog's structured facts alone still drive a deterministic, template-based fallback reading — Prolog is the reliable core; the LLM is a phrasing layer on top of it.

2. The Knowledge Base — `prolog/`

12 modules, about 4,600 lines total, all loaded once at process start via `consult()` in `backend/app/services/prolog_service.py`.

File	Lines	Purpose
<code>zodiac.pl</code>	110	Base facts: 12 signs, element, modality, ruling planet, traits, date ranges, symbols
<code>tarot.pl</code>	470	78 tarot cards, arcana, keywords, upright / reversed meanings, themes
<code>zodiac_profile.pl</code>	747	Extended personality profiles, birth-chart synthesis (Sun/Moon/Rising)
<code>card_selection.pl</code>	850	Context-aware card eligibility + priority scoring
<code>reading_analysis.pl</code>	751	Multi-card theme extraction, conflicts, dominant elements
<code>compatibility.pl</code>	145	Zodiac-pair compatibility scoring
<code>synastry.pl</code>	506	Deeper relationship analysis (element / modality / planet synergy)
<code>horoscope_rules.pl</code>	164	Daily theme + mood classification
<code>spread_rules.pl</code>	249	Tarot spread selection rules
<code>recommendation_rules.pl</code>	195	Question → category → spread recommendation
<code>reasoning.pl</code>	147	Orchestrator — combines other modules, builds explainability traces
<code>history_analysis.pl</code>	215	Aggregate stats & pattern insights across saved readings
<code>main.pl</code>	84	Entry point — <code>use_module</code> s everything, a few convenience wrappers

3. Core Prolog Techniques Used

This is the substantive part of the talk — six concrete techniques, each shown with real code from the project.

a) Facts as a relational database

`zodiac.pl` is pure facts — no rules needed. This is the simplest form of predicate logic: relations stored directly as ground clauses.

```
zodiac(aries).
element(aries, fire).
element(taurus, earth).
ruling_planet(aries, mars).
```

Querying `element(taurus, X)` binds `X = earth` purely by pattern-matching against the fact database — a clean way to explain **unification**.

b) Rules with unification + backtracking for classification

`compatibility.pl` is a great live-demo example. `zodiac_compatibility/3` has multiple clauses for the same head, each a different condition, tried top-down via backtracking:

```
zodiac_compatibility(Sign1, Sign2, high) :-
    element(Sign1, E), element(Sign2, E),
    modality(Sign1, M1), modality(Sign2, M2),
    compatible_modality(M1, M2).

zodiac_compatibility(Sign1, Sign2, high) :-
    element(Sign1, E1), element(Sign2, E2),
    compatible_element(E1, E2), E1 \= E2.

zodiac_compatibility(Sign1, Sign2, medium) :- ...
zodiac_compatibility(Sign1, Sign2, low) :- ...
```

This is literally an expert system: cascading if-this-then-that logic expressed **declaratively** instead of nested if/else in Python.

c) `findall/3` for aggregation

```
signs_of_element(Element, Signs) :-
    findall(Sign, element(Sign, Element), Signs).
```

Collects every solution to a goal into a list. Used throughout for "get all cards", "get all signs of an element", and history statistics.

d) SWI dicts as structured output

Instead of bare tuples, predicates build SWI-Prolog dicts (`tag{key: value, ... }`):

```
Info = zodiac_info{
  sign: Sign, element: Element, modality: Modality,
  ruling_planet: Planet, traits: Traits,
  date_range: DateRange, symbol: Symbol
}.
```

pyswip returns these as Python dicts almost directly — that's the glue that lets `prolog_service.py` do `result["Info"]["sign"]` cleanly, turning Prolog output straight into JSON for the API.

e) Priority scoring / ranked inference

`card_selection.pl` 's `card_priority/3` scores tarot cards 0–6 from three independent rule matches: zodiac affinity **+3**, category match **+2**, element match **+1**. The scoring logic is 100% Prolog; Python only sorts the already-scored results.

zodiac affinity +3

category match +2

element match +1

f) Explainability via generated "reasoning traces" — the best demo material

This is the most presentation-worthy part. `reasoning.pl` 's `generate_full_trace/3` doesn't just compute an answer — it builds a **list of trace_step dicts**, each showing which rule fired and what it concluded:

```
trace_step{
  rule:  format('element(~w, ~w)', [Sign, Element]),
  result: format('~w belongs to ~w element', [Sign, Element])
}
```

Every major module has a matching `*_reasoning_trace` predicate — `birth_chart_reasoning_trace`, `synastry_reasoning_trace`, `select_cards_reasoning`, and more. This is what lets the app show "here's why the system concluded X" — Prolog as an **explainable AI** system, not a black box.

IMPLEMENTATION DETAIL WORTH MENTIONING IF ASKED

`format(Template, Args)` inside a dict field isn't evaluated by SWI in that position — it stays an unevaluated compound term. `prolog_service.py` 's `_decode_format_term` helper parses that stringified term back into readable text in Python. A real gotcha the team hit and fixed.

4. Python ↔ Prolog Bridge

- **Library:** `pyswip` — Python bindings to SWI-Prolog, wrapping a single global `Prolog()` engine.
- **Query mechanism:** queries are built as strings and run via `prolog.query( ... )`, e.g.

```
f"zodiac_profile({_quote_atom(sign)}, Profile)"
```

- **Two call patterns:** `_first_result` (single solution) and `_safe_query` (all solutions) — both wrapped in `try/except`, so a failed or unloaded query returns `[]` / `None` rather than crashing the API.

Security: atom-injection prevention

Since queries are string-built from user input (question text, sign names), this is Prolog's cousin of SQL injection. The `_quote_atom` helper escapes backslashes **before** quotes:

```
escaped = text.replace("\\", "\\\").replace("'", "\\'")
return f'_{escaped}'
```

WHY ORDER MATTERS

Escaping only quotes (the naive approach) is unsafe: a value ending in a backslash immediately followed by a quote (e.g. `foo\'`) can escape the quoting and inject additional Prolog syntax into the query string built around it. Backslashes must be escaped first.

5. Suggested Demo Flow

1. Open with the architecture diagram — LLM never reasons, Prolog decides.
2. Live in `swipl: consult('main.pl').` then

```
?- zodiac_info(leo, X).  
?- zodiac_compatibility(aries, leo, L).
```

Press `;` to show backtracking finding alternate solutions.

3. Run a `*_reasoning_trace` query and show how each step maps 1:1 to what the frontend displays as "why".
4. Mention scale: ~4,600 lines of Prolog across 12 modules — a full expert system, not a toy rule file.

6. Anticipated Questions

Why Prolog instead of just if/else in Python?

Declarative rule composition, built-in multiple-clause backtracking search, a natural fit for a fact/rule knowledge base, and separation of concerns — reasoning logic lives in one auditable place instead of scattered across API handlers.

What happens if the LLM is down?

Prolog-derived facts alone still produce a deterministic, template-based reading or horoscope. The LLM is a phrasing layer, not a dependency for correctness.

How does Prolog output become an API response?

Predicates build SWI dicts (`tag{key: value}`); pyswip returns them as Python dicts; `prolog_service.py` converts fields to native Python types and returns plain dicts/JSON.

Is user input safe to embed in a Prolog query string?

Yes — every value goes through `_quote_atom`, which escapes backslashes before quotes to prevent atom-breakout injection.

How is "explainability" actually implemented, not just claimed?

Dedicated `*_reasoning_trace` predicates return an ordered list of `trace_step` dicts, each naming the exact rule that fired and the fact it derived — returned to the frontend as the reading's "why" trail.

AstroLogic · Prolog reasoning briefing · generated from the project's own source (prolog/, backend/app/services/prolog_service.py, docs/ai-integration.md)